

UNIT IV- PL/SQL PROGRAMMING

4.7 Database Triggers:

Use of Database Triggers,

How to apply database Triggers,

Types of Triggers,

Syntax for Creating Trigger,

Deleting Trigger.

Prepared By

Mr. Mohammad Ali

Lecturer (Selection Grade)

M. H. Saboo Siddik Polytechnic, Mumbai

TRIGGERS

- **A database trigger is a stored PL/SQL program unit associated with a specific database table.**
- ORACLE executes (fires) a database trigger automatically when a given SQL operation (like INSERT, UPDATE or DELETE) affects the table.
- Unlike a procedure, or a function, which must be invoked explicitly, **database triggers** are invoked **implicitly**.

TRIGGERS

Database triggers can be used to perform any of the following:

- Audit data modification
- Log events transparently
- Enforce complex business rules
- Derive column values automatically
- Implement complex security authorizations
- Maintain replicate tables
- Enforcing referential integrity
- Synchronous replication of tables
- Preventing invalid transactions

TRIGGERS

- We can associate up to **12 database triggers** with a given table.
- A database trigger has three parts:
 - a **triggering event**
 - an **optional trigger constraint** and
 - a **trigger action**.
- When an event occurs, a database trigger is fired, and an predefined PL/SQL block will perform the necessary action.

TRIGGERS - SYNTAX

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE|AFTER|INSTEAD OF}
{INSERT [OR] | UPDATE [OR] | DELETE} [OF col_name] ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
[WHEN condition]
DECLARE
Declaration statements
BEGIN
Executable statements
EXCEPTION
Exception-handling statements
END;
```

TRIGGERS

Where,

- **CREATE [OR REPLACE] TRIGGER trigger_name**: Creates or replaces an existing trigger with the *trigger_name*.
- **{BEFORE | AFTER | INSTEAD OF}**: This specifies when the trigger would be executed. The **INSTEAD OF** clause is used for creating trigger **on a view**.
- **{INSERT [OR] | UPDATE [OR] | DELETE}**: This specifies the DML operation.
- **[OF col_name]**: This specifies the column name that would be updated.
- **[ON table_name]**: This specifies the name of the table associated with the trigger.
- **[REFERENCING OLD AS o NEW AS n]**: This allows you to refer new and old values for various DML statements, like INSERT, UPDATE, and DELETE.
- **[FOR EACH ROW]**: This specifies a row level trigger, i.e., the trigger would be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.
- **WHEN (condition)**: This provides a condition for rows for which the trigger would fire. This clause is valid only for row level triggers.

TYPES OF TRIGGERS

Before and After Triggers

Since triggers occur because of events, they may be set to occur immediately before or after those events. Triggers may be called immediately **BEFORE** or **AFTER** the following events: **INSERT**, **UPDATE** and **DELETE**.

- The before/after options can be used to specify when the trigger body should be fired with respect to the triggering statement.
- If the user indicates a **BEFORE** option, then Oracle fires the trigger **before executing the triggering statement**.
- On the other hand, if an **AFTER** is used, Oracle fires the trigger **after executing the triggering statement**.
- **AFTER** row-level triggers are frequently used in auditing applications, since they do not fire until the row has been modified.

A trigger may be a **ROW** or **STATEMENT** type.

- If the statement **FOR EACH ROW** is present in the CREATE TRIGGER clause of a trigger, the trigger is a **row trigger**. A row trigger is fired for each row affected by an triggering statement.
- A **Statement Trigger**, however, is fired only once for the triggering statement, regardless of the number of rows affected by the triggering statement

TYPES OF TRIGGERS

Valid trigger types (possible combination of triggers)

- **Statement** (INSERT, DELETE, UPDATE), **Timing** (BEFORE, AFTER), **Level** (Row-level, Statement-level)
- The values for the statement, timing, and level determine the types of the triggers. There are total of 12 possible types of triggers: $3*2*2 = 12$

Combining Trigger Types

- Triggers for multiple INSERT, UPDATE, DELETE commands on a table can be combined into a single trigger (using **OR**), provided they are all at the same level (row-level or statement-level), e.g., **INSERT OR UPDATE OR DELETE**.
- However, we can not combine BEFORE or AFTER, e.g., BEFORE OR AFTER is illegal.

Statement-Level Trigger

- Statement-level triggers execute once for each transaction.
- For example, if a single transaction inserted 500 rows into the **emp** table, then a statement-level trigger on that table would only be executed once.
- Statement-level triggers therefore are not often used for *data-related* activities; they are normally used to enforce additional security measures on the types of transactions that may be performed on a table.
- Statement-level triggers are the default type of triggers created and are identified by **omitting** the **FOR EACH ROW** clause in the **CREATE TRIGGER** command.

Example: Statement Trigger

```
CREATE OR REPLACE TRIGGER mytrig1
BEFORE DELETE OR INSERT OR UPDATE ON emp
BEGIN
IF (TO_CHAR(SYSDATE, 'day') IN ('saturday', 'sunday')) OR
(TO_CHAR(SYSDATE, 'hh:mi') NOT BETWEEN '10:30' AND '18:30') THEN
RAISE_APPLICATION_ERROR(-20500, 'Table is secured.' || chr(10) || chr(9) ||
'You are not allowed to perform DML operations before 10.30 am and after 6.30
    pm.' || chr(10) ||
'You are not allowed to perform DML operations on Saturday and Sunday');
END IF;
END;
/
```

- The above example shows a trigger that limits the DML actions to the emp table to weekdays from 10.30 am to 6.30 pm.
- If a user tries to insert/update/delete a row in the EMP table, a warning message will be prompted.

Example: Statement Trigger

SQL> insert into emp

2 values(124,'ABC','CLERK',456,'15-JUL-1985',5000,0,20);

insert into emp

ERROR at line 1:

ORA-20500: Table is secured.

You are not allowed to perform DML operations before 10.30 am and after 6.30 pm

ORA-06512: at "SCOTT.MYTRIG1", line 4

ORA-04088: error during execution of trigger 'SCOTT.MYTRIG1'

insert into emp values(124,'ABC','CLERK',456,'15-JUL-1985',5000,0,20);

ROW-LEVEL TRIGGER

- Row-level triggers fires/executes once for each row affected by the triggering event such as **INSERT**, **UPDATE**, or **DELETE**.
- Row-level triggers are the most common type of triggers and they are useful for data-related activities such as data auditing and data validation.
- To create a new row-level trigger, we use the **CREATE TRIGGER** statement with the **FOR EACH ROW** clause.
- The features which are applicable to the statement-level triggers are also available with the row-level trigger:
- Check if the current event is INSERT, UPDATE, and DELETE using the **INSERTING** or **DELETING** or **UPDATING**.
- Most significant, row-level triggers allow us to track the **BEFORE** and **AFTER** values.

ROW-LEVEL TRIGGER

The :OLD & :NEW column values

Because row-level triggers execute within the context of a single row, we can access the old and new column values using the following syntax:

:OLD.column_name

:NEW.column_name

For example, this statement checks the value of the **sal** column to see if the **new sal** is greater than the **current sal**:

IF :NEW.sal > :OLD.sal THEN

-- carry an action

END;

This table illustrates the availability of :NEW and :OLD variables by the triggering event:

Triggering Event	:NEW	:OLD
INSERT	Yes	No
UPDATE	Yes	Yes
DELETE	No	yes

Modifying :OLD & :NEW values

A BEFORE row-level trigger can modify the new column values, but an AFTER row-level trigger cannot.

ROW-LEVEL TRIGGER

Performance consideration

- A row-level trigger fires each time a row is affected by a triggering event. For example, if we update 10000 rows in a table, the trigger will fire 10000 times, which potentially cause a performance issue.
- To specify a condition of when to fire the trigger, we can use the **WHEN** clause. For example, the following trigger only fires when we update **sal** of an employee with the **new sal** is greater than 10,000:

```
CREATE OR REPLACE TRIGGER trig1
BEFORE UPDATE OF sal
ON emp
FOR EACH ROW
WHEN NEW.sal > 10000;
...
```

- In some situations, using the condition in the **WHEN** can significantly improve the performance of the database.
- Note that we can use both **OLD** and **NEW** in the **WHEN** clause. In addition, we don't use a **colon (:)** as the prefix for these variables.

Example: ROW Trigger

```
CREATE OR REPLACE TRIGGER trig_grades
AFTER DELETE OR INSERT OR UPDATE ON grades
FOR EACH ROW
BEGIN
IF DELETING THEN
INSERT INTO xgrades(sname,c_code,marks,deldate)
VALUES(:old.sname,:old.c_code,:old.marks,sysdate);
ELSIF INSERTING THEN
    INSERT INTO ngrades(sname, c_code, marks, adddate)
    VALUES (:new.sname, :new.c_code, :new.marks, sysdate);
ELSIF UPDATING('marks') THEN
    INSERT INTO ugrades(sname, c_code, oldmarks, newmarks, up_date,
    user_responsible)
    VALUES (:old.sname, :old.c_code, :old.marks, :new.marks, sysdate, user);
END IF;
END;
/
```

OUTPUT:

```
Trigger created.
```

Example: ROW-LEVEL Trigger

- The previous trigger is used to keep track of all the transactions performed on the **grades** table.
- If any student detail is deleted, a new row containing the details of this student is stored in a table called **xgrades**.
- Similarly, if a new student detail is inserted, a new row is created in another table called **ngrades**, and
- Similarly, if **marks** of a student is updated, a new row is created in another table called **ugrades**.
- Note that we can specify the old and new values of an updated row by prefixing the column names with the **:OLD** and **:NEW** qualifiers.

Example: ROW Trigger

```
SQL> select * from xgrades;
```

no rows selected

```
SQL> delete from grades where sname='HABBAN';
```

2 rows deleted.

```
SQL> select * from xgrades;
```

SNAME	C_CODE	MARKS	DELDATE
HABBAN	DMS	60	10-OCT-20
HABBAN	CGR	65	10-OCT-20

```
SQL> _
```

```
SQL> select * from ngrades;
```

no rows selected

```
SQL> insert into grades values('HATIM','DMS',70);
```

1 row created.

```
SQL> select * from ngrades;
```

SNAME	C_CODE	MARKS	ADDDATE
HATIM	DMS	70	10-OCT-20

```
SQL> select * from ugrades;
```

no rows selected

```
SQL> update grades set marks=69 where sname='HATIM';
```

1 row updated.

```
SQL> select * from ugrades;
```

SNAME	C_CODE	OLDMARKS	NEWMARKS	UP_DATE	USER_RESPO
HATIM	DMS	70	69	10-OCT-20	SCOTT

```
SQL> select * from grades;
```

SNAME	C_CODE	MARKS
ANAM	DMS	70
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
CHAITANYA	DMS	60
CHAITANYA	CGR	62
HATIM	DMS	69

7 rows selected.

Example: ROW-LEVEL Trigger

- The following program creates a row level trigger for the **EMP** table that would fire for INSERT or UPDATE or DELETE operations performed on the **EMP** table. This trigger will display the salary difference between the old values and new values:

```
CREATE OR REPLACE TRIGGER Emp_salary_changes
BEFORE DELETE OR INSERT OR UPDATE ON emp
FOR EACH ROW
WHEN (NEW.empno > 0)
DECLARE
    sal_diff number;
BEGIN
    sal_diff := :NEW.sal - :OLD.sal;
    dbms_output.put_line('Old salary: ' || :OLD.sal);
    dbms_output.put_line('New salary: ' || :NEW.sal);
    dbms_output.put_line('Salary difference: ' || sal_diff);
END;
/
```

ENABLING, DISABLING, DROPPING TRIGGERS

SQL>ALTER TRIGGER trigger_name DISABLE;

SQL>ALTER TABLE table_name DISABLE ALL TRIGGERS;

To enable a trigger, which is disabled, we can use the following syntax:

SQL>ALTER TABLE table_name ENABLE trigger_name;

All triggers can be enabled for a specific table by using the following command

SQL> ALTER TABLE table_name ENABLE ALL TRIGGERS;

SQL> DROP TRIGGER trigger_name

Example: Trigger (As per manual's Exp. No 16)

```
CREATE OR REPLACE TRIGGER display_salary_changes BEFORE
DELETE OR INSERT OR UPDATE ON emp
FOR EACH ROW
WHEN (NEW.ID > 0)
DECLARE
    sal_diff number;
BEGIN
    sal_diff := :NEW.salary - :OLD.salary;
    dbms_output.put_line('Old salary: ' || :OLD.salary);
    dbms_output.put_line('New salary: ' || :NEW.salary);
    dbms_output.put_line('Salary difference: ' || sal_diff);
END;
/
Trigger created.
```

Triggering/using a Trigger(As per manual Exp. No. 16)

- Let us perform some DML operations on the EMP table. Here is one INSERT statement, which will create a new record in the table:
**INSERT INTO EMP (ID,NAME,AGE,ADDRESS,SALARY)
VALUES (7, 'Kriti', 22, 'HP', 7500.00);**
- When a record is created in EMP table, above create trigger **display_salary_changes** will be fired and it will display the following result:
Old salary:
New salary: 7500
Salary difference:

Triggering/using a Trigger(As per manual Exp. No. 16)

- Because this is a new record so old salary is not available and above result is coming as null. Now, let us perform one more DML operation on the EMP table. Here is one UPDATE statement, which will update an existing record in the table:

UPDATE EMP

SET salary = salary + 500

WHERE id = 2;

- When a record is updated in EMP table, above create trigger **display_salary_changes** will be fired and it will display the following result:

Old salary: 1500

New salary: 2000

Salary difference: 500